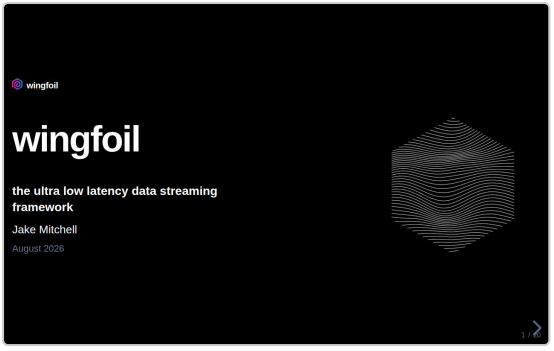
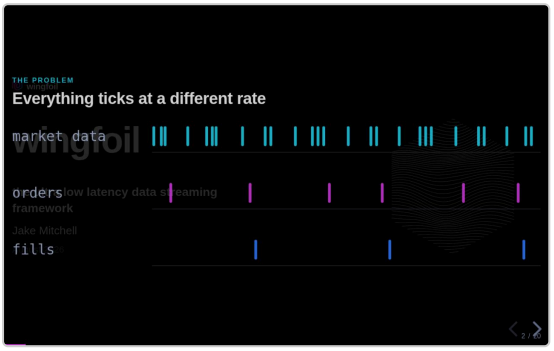
wingfoil — speaker notes

20 slides · printed fallback for the reveal.js speaker view (press S)



1 / 20

- Thanks for coming. Jake Mitchell. Open source project — **wingfoil**.
- Three parts:
- **1** — background: myself, the problem, and why you need wingfoil to solve it.
- **2** — **Nitro**. The feature we released last week. Why we wanted it. Why it meant rewriting the entire framework. The pattern we found, and what it delivered.
- **3** — what's next for the wingfoil project.
- **Me:**
  - 25 years in financial services.
  - Mostly as a quant at JPMorgan — building electronic trading systems for options: across interest rates, FX, precious metals.
  - Now at B2C2, working for their crypto options desk.
  - I learnt what the most effective patterns were for building these kinds of systems.
  - I wanted to bring them into the open source world. And I wanted to use **Rust** to build it.

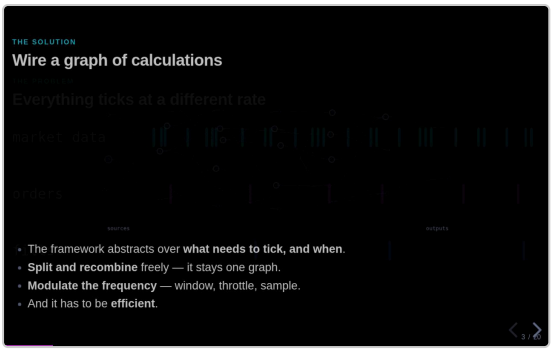


2 / 20

THE PROBLEM

Everything ticks at a different rate

- You're building something that reacts to information that is arriving at **different rates**.
- For example — market data ticking very frequently; orders and fills at a different frequency.
- This could be across tens of thousands of assets.
- And you need to react fast — at micro or even **nanosecond** time scales.



3 / 20

THE SOLUTION

Wire a graph of calculations

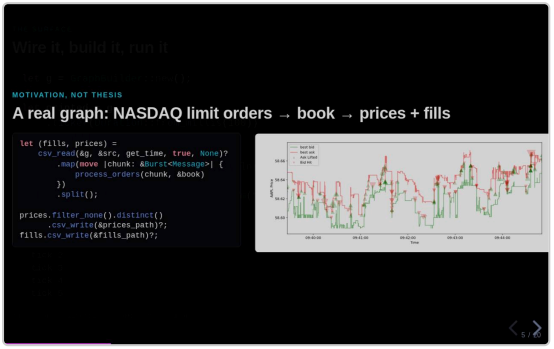
- Wire a graph of calculations.
- Have a framework that **abstracts over what needs to tick at what time**.
- That allows you to split and recombine.
- And to modulate their frequency — for example window or throttle.
- And is **efficient**.

4 / 20

THE SURFACE

Wire it, build it, run it

- This is what it looks like in practice.
- This example is a simple linear pipeline, with every node ticking on each engine cycle.

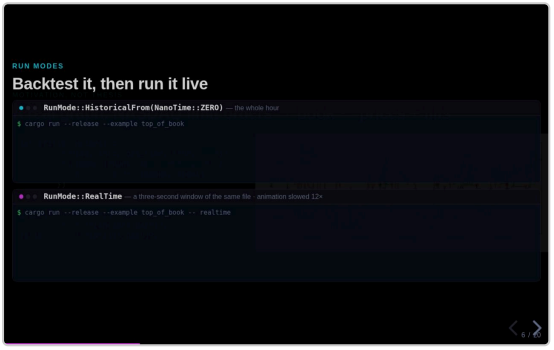


5 / 20

MOTIVATION, NOT THESIS

A real graph: NASDAQ limit orders → book → prices + fills

- Here is a more interesting example: a **matching engine**.
- Replay limit orders from file.
- Maintain an order book over time.
- Produce top-of-book prices and fills.
- **Three different frequencies of data** — many messages share a timestamp, top of book changes less often, trades are sparser still. Nothing is coalesced or dropped to make them agree.

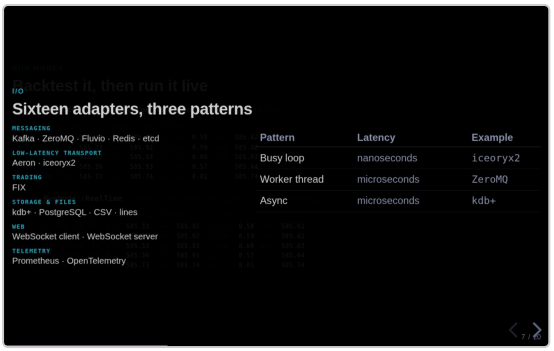


6 / 20

RUN MODES

Backtest it, then run it live

- Wingfoil supports **realtime** and **historical** modes.
- **Historical** [top] — development · unit tests · integration tests · back test.
- **Realtime** [bottom] — production.
- Same wiring. Same graph. **One line changes** — the run mode.

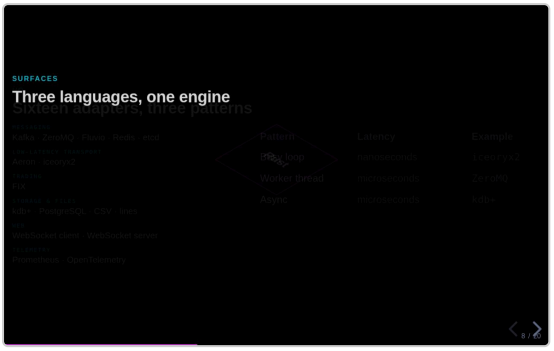


7 / 20

I/O

Sixteen adapters, three patterns

- We've built a bunch of I/O adapters — FIX, kdb+, iceoryx2, and more.
- They use these three I/O patterns:
  - **Busy spin** — for latency-critical, realtime use cases.
  - **Dedicated worker thread** with waker pattern.
  - **Async** — but only where you need it, at the graph edge. The graph itself is synchronous.
- You can **mix and match** — choose the pattern that works for your use case. The core stays synchronous either way, so **you are not locked into async**.



8 / 20

SURFACES

Three languages, one engine

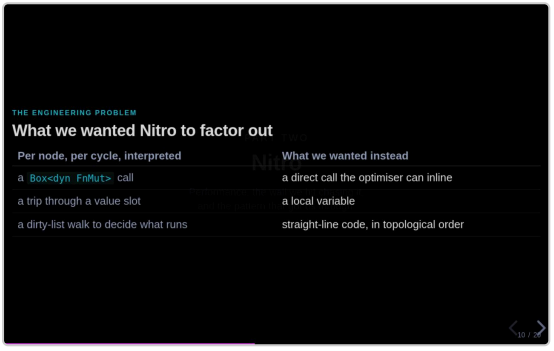
- **Rust** — the best language to work with.
- Also **Python** bindings, for those that have yet to make the jump.
- **TypeScript** integrations, to stream into the browser.

9 / 20

PART TWO

Nitro

- That's part one.
- Part two — **Nitro**.



10 / 20

THE ENGINEERING PROBLEM

What we wanted Nitro to factor out

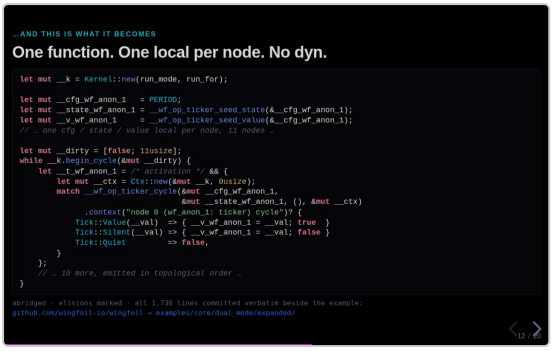
- A **dyn call** — an optimiser barrier at every node. LLVM can't see across a dyn boundary.
- A trip through a **value slot**.
- A **dirty-list walk** to work out what runs.

11 / 20

THE FRONT DOOR

You write the wiring once, in plain Rust

- **What we wanted to keep** — the same fluent ergonomics.
- Looks like familiar wiring code, but with the `nitro!` macro to expand it into something more efficient.

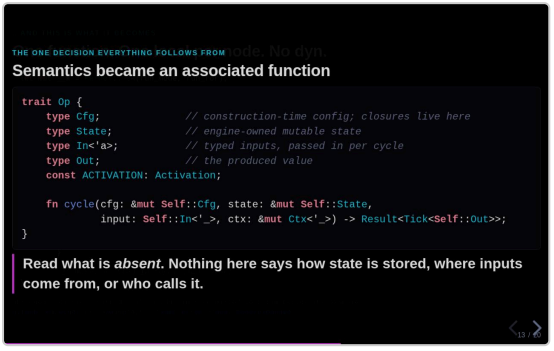


12 / 20

...AND THIS IS WHAT IT BECOMES

One function. One local per node. No dyn.

- One function. One local per node. **No dynamic dispatch.**

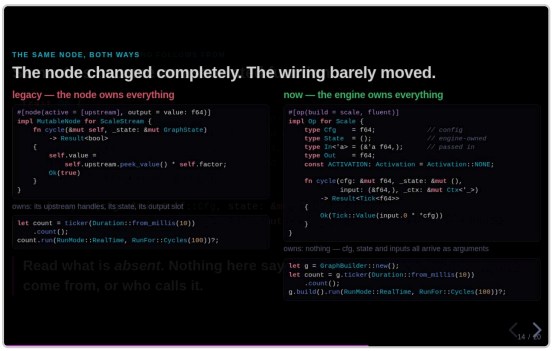


13 / 20

THE ONE DECISION EVERYTHING FOLLOWS FROM

Semantics became an associated function

- Four associated types and a function.
- **Notice what's missing.** Nothing says where state lives. Nothing says who calls it.
- It's a free function over explicit arguments.

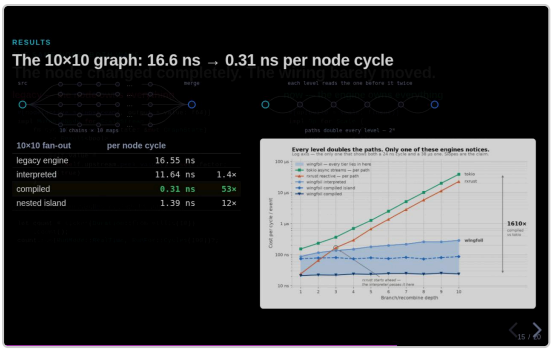


14 / 20

THE SAME NODE, BOTH WAYS

The node changed completely. The wiring barely moved.

- **Left: legacy.** The node owns references to upstream nodes. Owns its own state. `cycle` takes `&mut self`.
- **Right:** the same node as an **Op**. State lives on the outside.
- Same fluent wiring code.

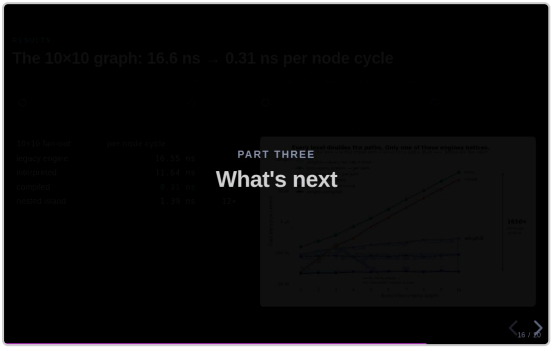


15 / 20

RESULTS

The 10×10 graph: 16.6 ns → 0.31 ns per node cycle

- These numbers were generated on low-powered VMs.
- **Two benchmarks here.**
- **[a] 10×10.** Wire a trivial graph 10 wide and 10 deep, with every node ticking on each cycle. Measure engine cycle time divided by the 100 nodes.
  - The old engine was around **16 ns** per node cycle; the new interpreted one **11.6 ns**.
  - Compiled went down to **0.31 ns** — less than a nanosecond per node cycle.
- **[b] Branch and recombine.** Take a source, split and recombine repeatedly to a given depth. Tokio and Rx perform badly on this because the number of node cycles **doubles at each level**.

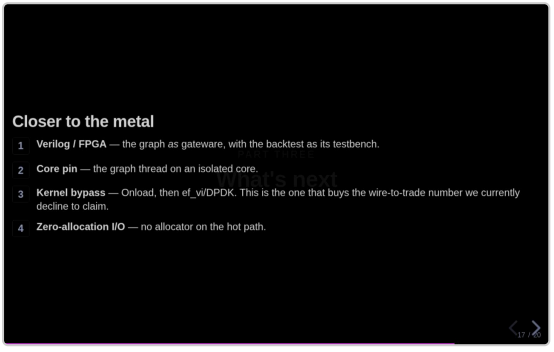


16 / 20

PART THREE

What's next

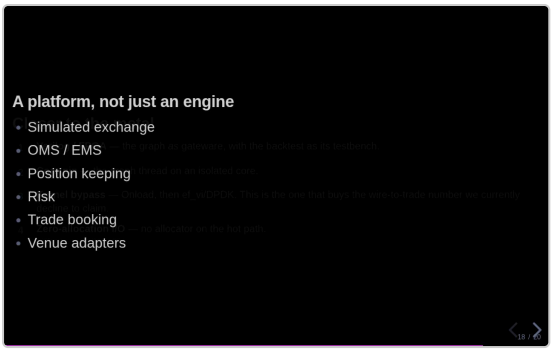
- That's Nitro.
- Last bit — where this goes.



17 / 20

Closer to the metal

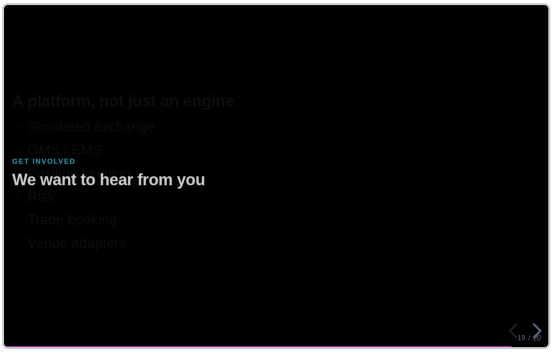
- **Verilog** — the graph as gateware, backtest as testbench. Exploratory. Don't oversell it.
- **Core pin** — graph thread on an isolated core. Working implementation sits in an example; needs promoting into the runtime.
- **Kernel bypass** — Onload, then raw ef_vi/DPDK. Gets us a wire-to-trade number, which **I won't claim yet because I haven't measured it**.
- **Zero allocation** on the I/O path.
- The point: **none of these change your wiring.** Same graph definition, further down. That's the payoff from part two.



18 / 20

A platform, not just an engine

- Simulated exchange
- OMS / EMS
- Position keeping
- Risk
- Trade booking
- Venue adapters

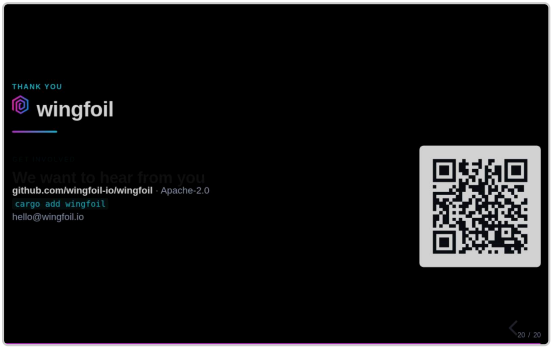


19 / 20

GET INVOLVED

We want to hear from you

- **Contributors.**
  - ~20 so far. I'd love more.
  - We've got some good first issues specified.
- We also want to **hear from you if you are interested in using wingfoil**.
  - Maybe for a trading firm, or building a bot to trade your personal account.



20 / 20

THANK YOU

wingfoil

- Thank you.
- Repo's on the QR.
